

# Actividad 7 — Videojuego web con HTML5 y Canvas

**Duración:** 50 min · **Modalidad:** individual · **Entrega:** repo GitHub público

## 1. Objetivos

- Crear un juego que corre **en el navegador** (sin instalar nada).
- Usar la etiqueta `<canvas>` para dibujar en JavaScript.
- Implementar el bucle de juego con `requestAnimationFrame`.
- Detectar teclado con **event listeners**.
- Juego: “Cazador de estrellas” (atrapar estrellas que caen).

## 2. Marco teórico

Los navegadores pueden dibujar gráficos con la etiqueta HTML `<canvas>`. El programa se escribe en **JavaScript** y el dibujo se hace con el “contexto 2D” (`getContext("2d")`).

La diferencia con Pygame es que acá no hay un `while` bloqueando el programa: usamos **`requestAnimationFrame`**, que le pide al navegador ejecutar nuestra función en el próximo cuadro. Así se arma el bucle de juego:

```
function loop() {  
  actualizar();  
  dibujar();  
  requestAnimationFrame(loop); // siguiente cuadro  
}  
loop();
```

La entrada se maneja con **eventos**: `keydown` (tecla apretada) y `keyup` (tecla soltada). Guardamos las teclas apretadas en un objeto para poder preguntar “¿está apretada la flecha izquierda?” en cada cuadro.

## 3. Paso a paso

```
cd mis-videojuegos  
mkdir clase_07_web
```

Escribí `clase_07_web/cazador.html` (un solo archivo que ya es un juego):

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="utf-8">  
  <title>Cazador de Estrellas</title>  
</head>  
<body>
```

```

<canvas id="juego" width="600" height="400"></canvas>
<script>
  const canvas = document.getElementById("juego");
  const ctx = canvas.getContext("2d");

  let x = 300;           // posición del cazador
  const velocidad = 5;
  let estrellaX = 100, estrellaY = 0, caida = 2;
  let puntos = 0;
  const teclas = {};

  document.addEventListener("keydown", e => teclas[e.key] = true);
  document.addEventListener("keyup", e => teclas[e.key] = false);

  function actualizar() {
    if (teclas["ArrowLeft"]) x -= velocidad;
    if (teclas["ArrowRight"]) x += velocidad;
    x = Math.max(20, Math.min(580, x)); // no salirse

    // La estrella cae
    estrellaY += caida;
    if (estrellaY > 400) { // se cayó sin atrapar
      estrellaY = 0;
      estrellaX = Math.random() * 580;
      puntos -= 1;
    }
    // ¿La atrapaste?
    if (Math.abs(estrellaX - x) < 30 && estrellaY > 360) {
      puntos += 1;
      estrellaY = 0;
      estrellaX = Math.random() * 580;
    }
  }

  function dibujar() {
    ctx.fillStyle = "#0a0a2e";
    ctx.fillRect(0, 0, 600, 400);

    ctx.fillStyle = "#ffd700";
    ctx.beginPath();
    ctx.arc(estrellaX, estrellaY, 10, 0, Math.PI * 2);
    ctx.fill();

    ctx.fillStyle = "#00ccff";
    ctx.fillRect(x - 25, 370, 50, 12);

    ctx.fillStyle = "white";
    ctx.font = "20px monospace";
    ctx.fillText("Puntos: " + puntos, 10, 24);
  }

```

```

    }

    function loop() {
        actualizar();
        dibujar();
        requestAnimationFrame(loop);
    }
    loop();
</script>
</body>
</html>

```

## 4. Probalo

Abrí el archivo con doble clic (se abre en tu navegador). También podés desde la terminal:

```
start clase_07_web/cazador.html
```

Importante: así como está, el archivo corre **solo** en tu máquina. Para que cualquiera en internet pueda jugarlo hay que publicarlo (actividad 16, GitHub Pages).

## 5. Actividad de cierre (para el commit)

1. Agregá que la velocidad de caída **aumente** con cada punto: `caida = 2 + puntos * 0.2`.
2. Mostrá un **mensaje** si los puntos llegan a -3 (“Perdiste”).
3. Agregá una segunda estrella (roja, vale 5 pero cae más rápido).
4. Commit y push:

```

git add .
git commit -m "Clase 07: cazador de estrellas en JavaScript"
git push

```

## 6. Usando IA en esta clase (prompts sugeridos)

**Prompt 1:** “¿Cuál es la diferencia entre un while infinito en Python y requestAnimationFrame en JavaScript para un bucle de juego?”

**Prompt 2:** “Agregá a mi cazador de estrellas una estrella roja que vale 5 puntos y cae el doble de rápido. Mi código: [pegá el código].”

**Prompt 3:** “¿Cómo hago para que el juego también se juegue tocando la pantalla en un celular (eventos touch)?”

**Prompt 4:** “Convertí el juego a un clon simple de ‘Frogger’: un personaje cruza una calle esquivando vehículos. Mostrame el código completo.”

## 7. Desafíos para seguir practicando

- Sonido con la API Audio de JavaScript.
- Que las estrellas aparezcan desde distintas posiciones y con distintas velocidades (una lista de objetos).
- Contador de tiempo en pantalla.
- Publicar el juego en GitHub Pages ya mismo (ver actividad 16).

## 8. Rúbrica de evaluación

| Criterio                       | Logrado                            | En proceso   | No logrado  |
|--------------------------------|------------------------------------|--------------|-------------|
| Corre en navegador             | Sin errores                        | Con errores  | No abre     |
| Bucle de juego                 | <code>requestAnimationFrame</code> | Otra técnica | No hay      |
| Teclado                        | Responsivo                         | Con fallas   | No funciona |
| Puntaje y dificultad creciente | Ambos                              | Uno          | Ninguno     |